

Netflix Movie Data Analysis & Recommendation System

Comprehensive Project Report

1. Introduction & Executive Summary

This project presents an end-to-end data science pipeline focusing on a dataset of approximately 9,800 movies. The primary objective is to extract actionable insights regarding movie popularity, genre trends over time, and audience voting behaviors. Furthermore, the project extends beyond traditional exploratory data analysis (EDA) by introducing machine learning models to predict success, a content-based recommendation engine for discoverability, an interactive dashboard for end-user exploration, and a computer vision module to analyze the visual aesthetics of movie posters.

By synthesizing these multi-disciplinary approaches (Data Analytics, Machine Learning, Web Deployment, and Computer Vision), this project serves as a robust blueprint for modern entertainment data analysis.

2. Dataset Overview & Initial Processing

The dataset, `mymoviedb.csv`, contains metadata for nearly 10,000 films. Key attributes include: - **Title & Release Date:** Identifiers and temporal data. - **Popularity & Vote Metrics:** Quantitative measures of user engagement (`Vote_Average`, `Vote_Count`, `Popularity`). - **Genre & Overview:** Qualitative attributes describing the movie's content.

Data Cleaning Procedures (`main_analysis.py`): 1. **Datetime Conversion:** Standardized `Release_Date` strings into datetime objects to extract the `Release_Year` for temporal analysis. 2. **Deduplication:** Aggregated duplicates by grouping records via `Title` and `Release_Year`, ensuring movie-level uniqueness. 3. **Data Type Handling:** Genre features were represented as comma-separated strings; these were parsed into discrete lists for multi-label processing.

3. Feature 1: Advanced Exploratory Data Analysis (EDA)

File: `main_analysis.py` The EDA phase uncovered several critical insights regarding how movies perform in the market.

3.1 Statistical Correlation Analysis

A correlation matrix was computed to understand the relationship between `Popularity`, `Vote_Count`, and `Vote_Average`. - **Finding:** A strong positive correlation exists between `Vote_Count` and `Popularity`. Movies that generate massive discussion and viewing volume inherently score higher on the popularity index. - **Finding:** `Vote_Average` (the actual rating from 1-10) showed weak linear correlation with `Popularity`. This indicates that a movie does not need to be a "critical masterpiece" to be highly popular; blockbuster appeal drives popularity more than critical acclaim.

3.2 Distribution Normalization

- The raw `Popularity` metric was heavily right-skewed, dominated by a few massive outliers (e.g., Marvel films).
- Applying a `log1p` (logarithmic) transformation successfully normalized the distribution, which is a crucial pre-processing step for linear modeling and statistical testing.

3.3 Genre Trends Over Time

- Analyzing the top 5 genres (Drama, Comedy, Action, Thriller, Adventure) from 1980 to the present revealed shifting consumer tastes.
 - While Drama and Comedy historically dominated market share in the 80s and 90s, Action and Thriller have seen a consistent, aggressive rise in proportion over the last two decades.
-

4. Feature 2: Machine Learning Classifier for Popularity

File: `main_analysis.py` To predict how a movie will perform based on its metadata, a supervised learning model was trained to predict the `Vote_Average` quartile category.

4.1 Methodology

1. **Target Engineering:** Vote_Average was bucketed into four distinct quartiles: not_popular, below_avg, average, and popular.
2. **Feature Engineering:** Genre was processed using MultiLabelBinarizer to create a multi-hot encoded matrix. The final feature set consisted of Vote_Count, Release_Year, and the binarized genres.
3. **Model Selection:** We compared Logistic Regression against a Random Forest Classifier.

4.2 Results & Evaluation

- **Logistic Regression:** Struggled to capture the complex, non-linear interactions between multiple genres and vote counts.
 - **Random Forest Classifier:** Achieved superior Accuracy and Macro F1-scores. The ensemble tree approach proved highly effective at understanding how specific combinations of genres (e.g., Action + Sci-Fi) interact with release eras to predict audience ratings.
 - The winning model was serialized via `joblib` into the `models/` directory for production deployment.
-

5. Feature 3: Content-Based Recommendation Engine

File: `main_analysis.py` & `app.py` Discoverability is a major challenge in large movie catalogs. We developed a content-based filtering system to solve this.

5.1 Algorithmic Approach

- **Textual Similarity:** Utilized `TfidfVectorizer` (Term Frequency-Inverse Document Frequency) to convert the Overview (plot summary) of each movie into numerical vectors. This naturally down-weights common words and emphasizes unique plot identifiers.
- **Categorical Similarity:** Genres were tokenized and similarly vectorized.
- **Hybrid Matrix:** The TF-IDF matrix and the Genre matrix were horizontally stacked using `scipy.sparse.hstack`. A weighting factor was applied to ensure Genres heavily influenced the final recommendation.
- **Distance Metric:** `cosine_similarity` was computed across the matrix to find the nearest neighbors for any given movie.

5.2 Fault Tolerance

- **Fuzzy Matching:** Integrated Python's `difflib.get_close_matches` to gracefully handle user typos. If a user queries "Spidr Man", the system resolves it to "Spider-Man" before computing similarities, vastly improving the user experience.
-

6. Feature 4: Interactive Web Dashboard

File: `app.py` To make the data accessible to non-technical stakeholders, an interactive web application was deployed using `Streamlit`.

- **Dynamic Filtering:** Users can filter the entire 9,800 movie dataset by combining multiple genres, selecting release year ranges via sliders, and setting minimum vote count thresholds.
 - **Performance Optimization:** The application leverages `@st.cache_data` to ensure the dataset and models are only loaded into memory once, providing instantaneous UI updates.
 - **Visual Analytics:** The dashboard features real-time KPI cards (Total Movies, Average Popularity) alongside dynamic Plotly charts that update based on user filters.
-

7. Feature 5: Computer Vision - Poster Color Analysis

File: `main_analysis.py` Moving beyond text and metadata, we introduced a Computer Vision module to analyze the visual aesthetics of movie marketing.

7.1 Data Acquisition

- Dynamically fetched movie poster images over HTTP using the `requests` library.
- Sampled ~150 movies heavily weighted toward the top 5 genres to ensure statistical significance without requiring immense compute time.

7.2 Unsupervised Learning (K-Means Clustering)

- Image arrays were reshaped into 2D pixel lists (RGB values).
- `sklearn.cluster.KMeans` (with `k=1`) was applied to calculate the mathematical centroid (dominant color) of each

- poster.
- Perceived brightness was calculated using the standard luminosity formula: $0.299 * R + 0.587 * G + 0.114 * B$.

7.3 Insights

- Visual data confirmed psychological color theory in marketing:
 - **Comedy & Animation**: Posters exhibited significantly higher average brightness and variance, utilizing vibrant color palettes.
 - **Thriller, Crime, & Horror**: Posters skewed heavily toward the lower end of the luminosity spectrum, relying on dark, shadowy aesthetics to convey tone.
-

8. Engineering Best Practices & CI/CD

To ensure software reliability, the project was fortified with standard engineering practices: - **Unit Testing**: Developed a comprehensive test suite using `pytest`. The tests validate data cleaning algorithms (e.g., verifying quartile bucketing logic) and confirm the mathematical accuracy of the recommender system. - **Continuous Integration**: Configured a GitHub Actions workflow (`ci.yml`) to automatically trigger the test suite on every code push, ensuring future modifications do not break core functionality.

9. Conclusion

This project successfully transformed a raw CSV of movie metadata into a highly interactive, intelligent system. By bridging the gap between deep statistical EDA, supervised and unsupervised machine learning, and robust software engineering, the resulting pipeline offers a comprehensive toolset for analyzing and predicting entertainment industry trends.